Representation Learning for Rankings

Lecture 5: Dense Retrieval — From Bi-Encoders to Late Interaction

Avishek Anand

Information Retrieval

March 2, 2026

Outline

1. Recap & Motivation — From Re-ranking to Retrieval
2. Contrastive Learning for Retrieval
3. Dense Passage Retrieval (DPR)
4. ColBERT: Late Interaction
5. Hard Negatives Mining
6. Evaluation & The Zero-Shot Challenge
7. Retrieval-Augmented Generation (RAG)
8. Connections and Summary
9. Summary and Assignment

Section 1

Recap & Motivation — From Re-ranking to Retrieval

Recap: Where We Are

What We've Built So Far

- **Lecture 2:** Inverted indexes and ANN indexes (IVF+PQ, HNSW)
- **Lecture 5:** BM25, Language Models, RM3 for scoring and expansion
- **Lecture 6:** BERT cross-encoders for re-ranking (monoBERT, monoT5)

The Remaining Problem

Cross-encoders are **too slow for retrieval**:

- Re-ranking 100 docs: 0.5s ✓ Full corpus (8.8M): 12+ hours ✗
- BM25 misses semantic matches: "automobile issues" vs. "car problems"
- Bounded Recall: if retriever misses a relevant doc, the cross-encoder can **never** recover it

Today's Goal

The Missing Piece

Combine the **semantic understanding** of BERT with the **retrieval speed** of ANN indexes from Lecture 2.

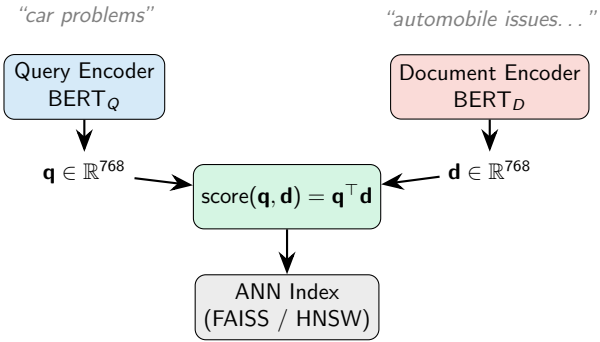
Key Idea: The Bi-Encoder

1. Encode all documents **offline** with BERT $\rightarrow$ dense vectors
2. Store in ANN index (IVF+PQ from Lecture 2!)
3. At query time: encode query with BERT $\rightarrow$ ANN search $\rightarrow$ top-k

The Trade-off

Bi-encoders are **less accurate** than cross-encoders (no query-document interaction) but **fast enough for full-corpus retrieval**.

The Dense Retrieval Pipeline



Offline (once):

- Encode all 8.8M passages
- Build ANN index
- ∼hours on GPU

Online (per query):

- Encode query: ∼5 ms
- ANN search: ∼10 ms
- Total: ∼15 ms ✓

Cross-Encoder vs. Bi-Encoder: Architecture

Cross-Encoder: BERT([CLS] query [SEP] doc [SEP])

gets relevance score from output layer

Bi-Encoder: $\text{BERT}_Q(\text{query})$ $\text{BERT}_D(\text{doc})$

have to compute $\mathbf{q}^\top \mathbf{d}$

Key Question

How do we **train** the bi-encoder so the dot product captures relevance?

Cross-Encoder vs. Bi-Encoder: Comparison

	Cross-Encoder	**Bi-Encoder**
Query-doc interaction	Full (all layers)	None (dot product only)
Pre-compute doc embeddings	✗ No	✓ Yes (offline)
Use for retrieval	✗ No (too slow)	✓ Yes (with ANN)
Accuracy	Higher	Lower
Speed at inference	$\sim$5 ms/pair	$\sim$15 ms total

Section 2

Contrastive Learning for Retrieval

Contrastive Learning: Core Principle

Objective

Learn embeddings such that:

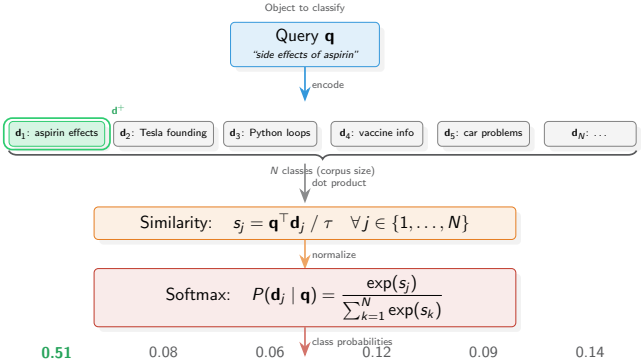
$$\text{score}(\mathbf{q}, \mathbf{d}^+) \gg \text{score}(\mathbf{q}, \mathbf{d}^-)$$

for relevant pair $(\mathbf{q}, \mathbf{d}^+)$ and irrelevant pair $(\mathbf{q}, \mathbf{d}^-)$.

Intuition: The embedding space is a **map of meaning**.

- "car problems" and "automobile issues" $\rightarrow$ nearby points
- "car problems" and "car racing" $\rightarrow$ far apart

InfoNCE: Noise Contrastive Estimation — The Classification View



Key Insight

InfoNCE frames retrieval as N-**way classification**: given query $\mathbf{q}$, classify which of the N documents in the corpus is the relevant one, using dot-product similarity as the logit.

Theoretical InfoNCE: The Global View

The Goal: Contrastive Estimation

Given query $\mathbf{q}$ and its relevant document $\mathbf{d}^+$, we want to maximize the probability:

$$P(\mathbf{d}^+|\mathbf{q}) = \frac{\exp(\mathbf{q}^\top \mathbf{d}^+/\tau)}{\sum_{\mathbf{d}\in\mathcal{D}} \exp(\mathbf{q}^\top \mathbf{d}/\tau)}$$

where $\mathcal{D}$ is the **entire corpus** (e.g., 8.8M passages).

The InfoNCE Loss

The loss minimizes the negative log-probability of the positive document:

$$\mathcal{L}_{\text{InfoNCE}} = -\log P(\mathbf{d}^+|\mathbf{q}) = -\log \frac{\exp(\mathbf{q}^\top \mathbf{d}^+/\tau)}{\sum_{\mathbf{d}\in\mathcal{D}} \exp(\mathbf{q}^\top \mathbf{d}/\tau)}$$

InfoNCE: The Computational Bottleneck

The InfoNCE Loss

The loss minimizes the negative log-probability of the positive document:

$$\mathcal{L}_{\text{InfoNCE}} = -\log P(\mathbf{d}^+|\mathbf{q}) = -\log \frac{\exp(\mathbf{q}^\top \mathbf{d}^+/\tau)}{\sum_{\mathbf{d}\in\mathcal{D}} \exp(\mathbf{q}^\top \mathbf{d}/\tau)}$$

The Computational Bottleneck

The denominator requires computing the dot product for **every single document** in the corpus for every gradient step.

- 8.8M documents $\times$ 128 queries per batch $\approx$ 1.1 Billion dot products.
- Numerically and computationally **impossible** to do directly.

The InfoNCE Loss: Practical Approximation

In-Batch Contrastive Loss

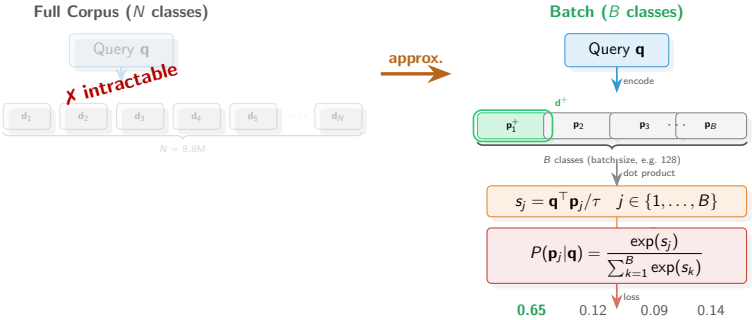
We approximate the global sum with a sum over the current **training batch** of size B:

$$\mathcal{L} = -\log \frac{\exp\left(\mathbf{q}_i^\top \mathbf{p}_i^+ / \tau\right)}{\sum_{j=1}^{B} \exp\left(\mathbf{q}_i^\top \mathbf{p}_j / \tau\right)}$$

Key components:

- B: batch size — provides $B - 1$ **free** negatives per query
- τ: temperature — controls sharpness of the softmax

InfoNCE: From N Classes to B Classes



Batch Approximation

Instead of classifying over the **entire corpus** (N classes), we classify over only the **batch** (B classes). Each batch provides $B-1$ in-batch negatives for free.

Section 3

Dense Passage Retrieval (DPR)

DPR: Architecture (Karpukhin et al., 2020)

"who founded wikipedia"

BERT_Q

$\mathbf{q} = \texttt{[CLS]}_Q \in \mathbb{R}^{768}$

"Wikipedia was founded by. . . "

BERT_D

$\mathbf{p} = \texttt{[CLS]}_D \in \mathbb{R}^{768}$

Key Design Choices

- **Representation:** `[CLS]` token output — single vector per text
- **Similarity:** $\text{score}(\mathbf{q}, \mathbf{p}) = \mathbf{q}^\top \mathbf{p}$ (dot product)

DPR: Training Recipe — Loss & Negatives

Loss: InfoNCE with in-batch negatives

$$\mathcal{L} = -\log \frac{e^{\mathbf{q}_i^\top \mathbf{p}_i^+/\tau}}{\sum_{j=1}^{B} e^{\mathbf{q}_i^\top \mathbf{p}_j/\tau}}$$

Negatives per query:

1. $B - 1$ in-batch negatives (free!)
2. 1 BM25 hard negative (precomputed)

Key Insight

Mixing **one** BM25 hard negative with in-batch random negatives is surprisingly effective. The BM25 negative is lexically similar but not relevant — forces the model beyond surface matching.

DPR: Why BM25 Hard Negatives?

Query: "side effects of aspirin"

Random negative: "The capital of France is Paris..."

- $\mathbf{q}^\top \mathbf{d}^- \approx 0.05$ — trivially easy, **no learning signal**

BM25 hard negative: "Aspirin was invented by Bayer in 1899..."

- Contains "aspirin" $\rightarrow$ BM25 ranks it high
- But **not about side effects** $\rightarrow$ not relevant
- $\mathbf{q}^\top \mathbf{d}^- \approx 0.6$ — confusing, **strong learning signal!**

The Principle

The model must learn: "just because a passage mentions aspirin doesn't mean it's about side effects." This is **exactly** what BM25 cannot do.

DPR: Results & Significance

Method	NQ Top-20 Recall	MS MARCO MRR@10	Params
BM25	59.1	18.7	—
DPR	**78.4**	**31.1**	220M

Why DPR Matters

- First system to conclusively show **dense** > **sparse** for open-domain QA
- +19pp top-20 recall over BM25 on Natural Questions
- Enabled the modern "retriever + reader" paradigm

But...

A single 768-dim vector for the entire document? That's a severe **information bottleneck**. Can we do better?

Section 4

ColBERT: Late Interaction

The Single-Vector Bottleneck

Problem with DPR

The **entire** document is compressed into a single 768-dim vector.

"Wikipedia was founded by Jimmy Wales and Larry Sanger in January 2001"

↓ information bottleneck

$\mathbf{d} \in \mathbb{R}^{768}$

Consequences:

- Cannot capture fine-grained token-level alignments
- "**Who** founded Wikipedia?" vs. "**When** was Wikipedia founded?" $\rightarrow$ similar scores!
- Long documents lose information

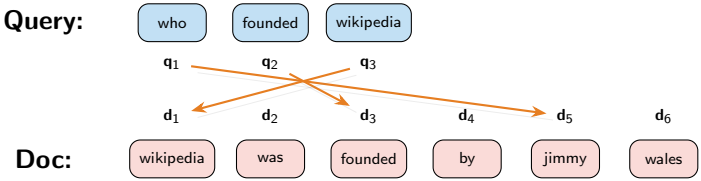
Solution

Keep **all** token embeddings, not just `[CLS]`. Compare at the token level.

ColBERT: Architecture (Khattab & Zaharia, 2020)

Key Idea

Encode query and document **separately** (like bi-encoder), but compare at the **token level** using MaxSim.



For each $\mathbf{q}_i$:
find best $\mathbf{d}_j$

Late Interaction

Documents are encoded **offline** (token-level embeddings stored). The "interaction" happens **late** — only at query time via MaxSim.

ColBERT: MaxSim Scoring

MaxSim Definition

$$\text{MaxSim}(\mathbf{q}, \mathbf{d}) = \sum_{i=1}^{|\mathbf{q}|} \max_{j=1}^{|\mathbf{d}|} \cos(\mathbf{q}_i, \mathbf{d}_j)$$

Step by step:

1. For each query token $\mathbf{q}_i$, find the document token $\mathbf{d}_j$ with highest cosine similarity
2. Take that maximum similarity
3. Sum across all query tokens

Properties:

- **Asymmetric:** Every query token must find a match; document tokens can be ignored
- **Decomposable:** Document embeddings precomputed and stored offline
- Token embeddings: $\mathbf{q}_i, \mathbf{d}_j \in \mathbb{R}^{128}$ (projected down from 768)

Worked Example: MaxSim Scoring

Query: "who founded wikipedia" (3 tokens)
Document: "wikipedia was founded by jimmy wales" (6 tokens)

	wikipedia	*was*	*founded*	*by*	*jimmy*	*wales*
who	0.12	0.08	0.15	0.10	**0.72**	0.45
founded	0.20	0.05	**0.95**	0.10	0.12	0.08
wikipedia	**0.98**	0.06	0.18	0.04	0.10	0.15

$$\text{MaxSim} = \underbrace{0.72}_{\text{who}\to\text{jimmy}} + \underbrace{0.95}_{\text{founded}\to\text{founded}} + \underbrace{0.98}_{\text{wikipedia}\to\text{wikipedia}} = \mathbf{2.65}$$

Notice

"who" aligns with "jimmy" (the answer entity) — the model learns **semantic alignment**, not just exact matching.

ColBERT: Efficiency vs. Accuracy

	DPR (Bi-Enc)	ColBERT	Cross-Encoder
Interaction	None (dot prod)	Late (MaxSim)	Full (cross-attn)
Pre-compute docs	✓ (1 vec)	✓ (n vecs)	✗
Use for retrieval	✓ With ANN	✓ With ANN	✗ Too slow
Speedup vs. CE	$\sim$1000$\times$	$\sim$100$\times$	1$\times$
MS MARCO MRR@10	31.1	36.0	39.0
Storage per doc	$768 \times 4\text{B} = 3$ KB	$128 \times 128 \times 4\text{B} \approx 64$ KB	N/A

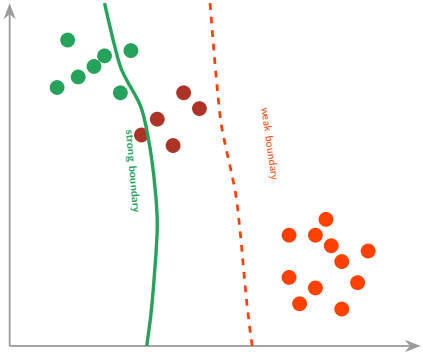
Sweet Spot

ColBERT gives near-cross-encoder accuracy at bi-encoder-like speed. Trade-off: **much higher storage** (one vector per token, not per document).

Section 5

Hard Negatives Mining

Hard vs. Easy Negatives: Why It Matters



Positives ($\mathbf{d}^+$)
Hard negatives
Easy negatives

InfoNCE Loss:

$$\mathcal{L}(\mathbf{q}_i, \mathbf{p}_i^+, \mathbf{p}_{i,1}^-, \cdots, \mathbf{p}_{i,n}^-)$$

$$= -\log \frac{e^{\text{sim}(\mathbf{q}_i, \mathbf{p}_i^+)}}{e^{\text{sim}(\mathbf{q}_i, \mathbf{p}_i^+)} + \sum_{j=1}^{n} e^{\text{sim}(\mathbf{q}_i, \mathbf{p}_{i,j}^-)}}$$

Hard negatives are close to the positives in embedding space $\rightarrow$ large $\text{sim}(\mathbf{q}, \mathbf{p}^-)$.

This makes the denominator large, the loss **harder to minimize**, and gradients **more informative**.

The Evolving Encoder and Negative Quality

Key Insight: What Counts as "Hard" Changes Over Training

The encoder's understanding of relevance **evolves** across training epochs. This changes which negatives are informative:

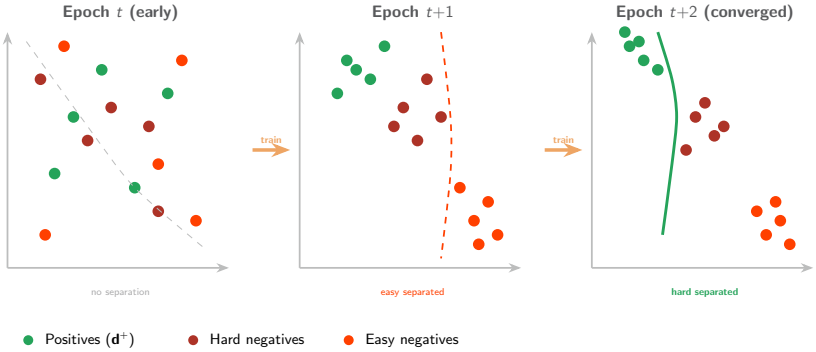
Early training: The encoder has no notion of relevance.

- BM25 negatives (lexically similar but irrelevant) are **not yet hard** — the model cannot even distinguish basic relevance.
- Random negatives provide sufficient learning signal.

Later training: The encoder understands basic relevance.

- Random negatives become **trivially easy** — zero gradient.
- BM25 negatives now become **genuinely hard** — the model must learn to distinguish lexical overlap from true relevance.

The Evolving Encoder: Representations Shift Over Training



Implication

The optimal negative sampling strategy is **not fixed** — it should adapt to the encoder's current capability.

From Static to Dynamic Negatives

Static Negatives (e.g., BM25)

Negatives are chosen **once, before training** and remain fixed:

- Cheap to compute (run BM25 once).
- Eventually become too easy as the encoder improves.

Dynamic Negatives (e.g., ANCE)

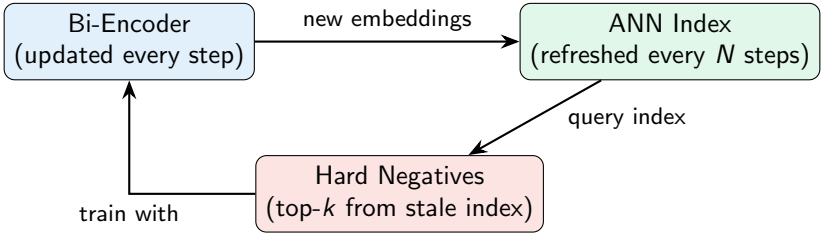
Negatives are defined by the **current encoder state**:

- A **hard negative** = a document the model **currently** scores high, but is **not** the ground-truth relevant document.
- These are the most confusing examples *right now* — maximally informative gradients.
- Requires periodically re-encoding the corpus and refreshing the index.

ANCE: Asynchronous Index Refresh

Core Innovation

Maintain an ANN index during training. Use the model's **own** top-k nearest passages as hard negatives.



ANCE: Results

Method	NQ Top-20 R	MS MARCO MRR@10	Negatives
BM25	59.1	18.7	—
DPR (BM25 neg.)	78.4	31.1	Static / lexical
ANCE	**81.9**	**33.0**	Dynamic / semantic

Key Takeaway

The **only** difference between DPR and ANCE is the negative sampling strategy. Semantic hard negatives from the model's own index are strictly better than BM25 hard negatives.

Implication

Negative sampling is a **first-order concern** in dense retrieval.
The model and architecture stay the same — the negatives matter most.

Section 6

Evaluation & The Zero-Shot Challenge

Key Benchmarks

Benchmark	Corpus	Queries	Primary Metric
MS MARCO	8.8M passages	500K train	MRR@10
Natural Questions	21M passages	3.5K test	Top-20 Recall
BEIR (18 tasks)	Varies	Varies	nDCG@10 (zero-shot)
TREC-DL 2019/20	8.8M passages	43/54	nDCG@10

Critical Distinction

- MS MARCO / NQ: **In-domain** (train & test same distribution)
- BEIR: **Zero-shot** (train on MS MARCO, test on 18 diverse tasks)
- Dense methods excel in-domain but often struggle on BEIR!

The Zero-Shot Challenge: BEIR

	BM25	DPR	ColBERT	ANCE	Contriever
BEIR Avg nDCG@10	47.3	33.2	41.5	37.7	48.5

Surprising Finding

BM25 outperforms most dense models on zero-shot BEIR!

Why?

- Dense models overfit to MS MARCO query structure.
- BM25 is distribution-agnostic (just term statistics).
- BEIR includes diverse tasks (fact-checking, etc.) where semantic matching isn't always enough.

Solving Zero-Shot: Contriever

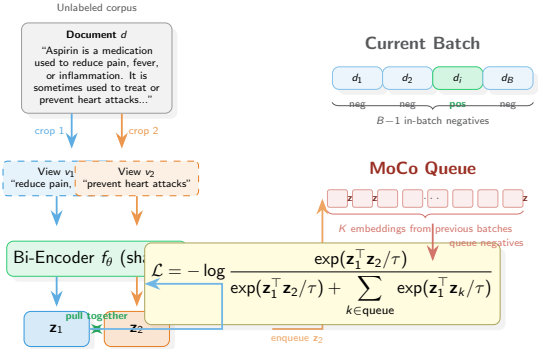
Contriever (Izacard et al., 2022)

Instead of learning retrieval from **manual relevance labels**, how can we learn from **raw text alone** using contrastive learning.

Result

Contriever (48.5) finally beats BM25 (47.3) on average across BEIR, showing that domain-agnostic dense retrieval is possible.

How Contriever Solves the Zero-Shot Problem



Why Contriever Works (and What It Does Not Fix)

Why it works

- Learns **domain-agnostic semantic similarity**.
- Trained on **diverse web text**, not one supervised benchmark.

Limits

- Still not always best at **top-rank precision** (*nDCG@10*).
- Long, argument-heavy documents remain hard (e.g., Tóuché).

Teaching takeaway

Contriever shows that dense retrieval can generalize **without labels**:

$$\text{better pretraining} > \text{more MS MARCO supervision}$$

for zero-shot transfer.

Hybrid Retrieval: Dense + Sparse

Key Finding

Combining BM25 and dense retrieval **outperforms either alone**.

$$\text{score}_{\text{hybrid}}(q, d) = \alpha \cdot \text{score}_{\text{BM25}}(q, d) + (1 - \alpha) \cdot \text{score}_{\text{dense}}(q, d)$$

Method	NQ Top-20 Recall
BM25 alone	62.9
DPR alone	78.4
BM25 + DPR (hybrid)	**82.6**

Why Hybrid Works

Sparse captures **exact matches** (named entities, rare terms). Dense captures **semantic similarity** (paraphrases). They are complementary, not competing.

Section 7

Retrieval-Augmented Generation (RAG)

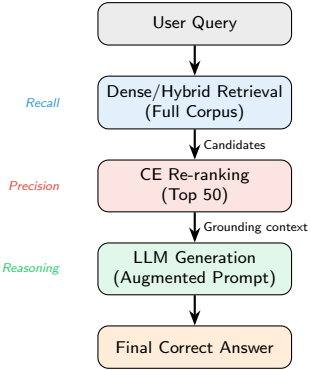
Beyond Ranking: Retrieval-Augmented Generation (RAG)

The Paradigm Shift

In modern AI applications, retrieval is not the final step. It is the **foundation** for Large Language Models (LLMs).

- **Problem:** LLMs hallucinate and have a fixed knowledge cutoff.
- **Solution:** Retrieve relevant, up-to-date passages and provide them as **grounding context** in the model's prompt.
- **Impact:** Retrieval expands the model's memory and enables it to cite its sources.

The Modern IR Pipeline with RAG



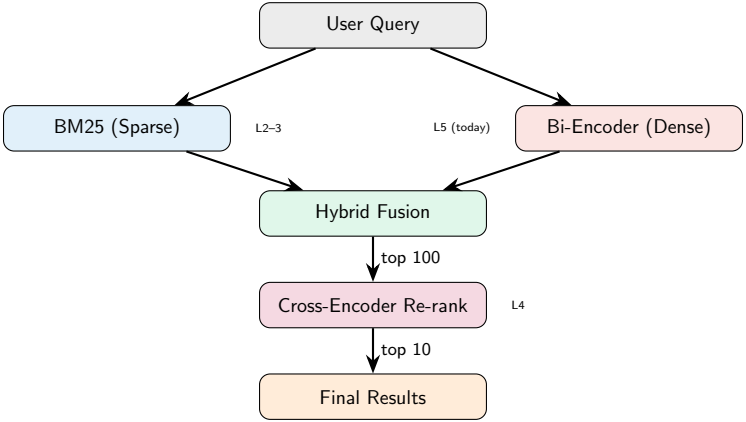
Retrieval as the "Eyes" of the LLM

If the retriever (bi-encoder) or re-ranker (cross-encoder) fails, the generator **cannot** produce a factual answer. High-quality IR is the bottleneck for RAG accuracy.

Section 8

Connections and Summary

The Complete Modern Retrieval Stack



Each lecture adds a component. Dense retrieval (today) gives us semantic first-stage retrieval — the missing piece from Lecture 4.

Section 9

Summary and Assignment

Key Takeaways

1. Bi-Encoders Enable Semantic Retrieval at Scale

Encode documents offline, search with ANN indexes from Lecture 2. Solves the vocabulary mismatch problem that BM25 cannot.

2. Contrastive Learning Is the Training Paradigm

InfoNCE loss with in-batch negatives. Larger batches = more negatives = better training. Temperature τ is a critical hyperparameter.

3. DPR $\rightarrow$ ColBERT $\rightarrow$ ANCE: Improving Dense Retrieval

DPR: single-vector bi-encoder. ColBERT: token-level late interaction. ANCE: dynamic hard negatives via asynchronous index refresh.

4. Hybrid Retrieval Is the Practical Standard

Dense + sparse complementary. BM25 for exact match, dense for semantics. Cross-encoder on top for final precision.

Questions?

Questions?

Preview: Next class we solve the *Recall Ceiling* with **Dense Retrieval**.